실습 9차시

Skills 기초: 나만의 전문가

학습 목표

- ✓ Skill의 구조(이름, 설명, 트리거, 본문)를 이해하고 기본 Skill을 작성할 수 있어요
- ✓ 작성한 Skill을 Claude Code에 등록하고 실행할 수 있어요
- ✓ Skill이 정상 동작하는지 테스트하고 문제를 진단할 수 있어요

? 핵심 질문

MCP가 Claude에게 "도구"를 줬다면, 어떻게 하면 Claude에게 "전문 지식"을 줄 수 있을까요? 매번 같은 설명을 반복하지 않고, 한 번 정의해두면 자동으로 불러와지는 방법이 있을까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 여러분만의 커밋 스킬이 완성돼요. `/commit`이라고 입력하면 Claude가 변경된 파일을 분석하고, 적절한 커밋 메시지를 작성하고, 확인을 거쳐 커밋까지 해줘요. 이 과정 전체가 하나의 스킬로 자동화되는 거예요.

배경지식: Skills란 무엇인가

이론편에서 MCP와 Skills의 관계를 배웠어요. MCP가 USB 포트라면, Skills는 그 포트에 연결되는 전문 앱이에요.

Skill은 Claude Code에게 특정 작업의 수행 방법을 가르치는 마크다운 파일이에요. CLAUDE.md가 "이 프로젝트는 이렇게 생겼어"라는 정보를 주는 거라면, Skill은 "이런 요청이 오면 이렇게 처리해"라는 절차를 주는 거예요.

핵심 차이를 정리하면 이래요.

- **CLAUDE.md**: 프로젝트의 맥락 정보 (수동적, 항상 로드)
- **Skills**: 특정 작업의 수행 절차 (능동적, 요청 시 로드)

준비물

- Claude Code (이전 실습에서 설정 완료)
- Git이 초기화된 프로젝트 폴더
- 텍스트 에디터

1. Skills 폴더 구조 이해

1 **Skills가 어디에 저장되고 어떤 구조인지 이해해요**

Skills 파일은 `.claude/commands/` 폴더 안에 마크다운 파일로 저장돼요. 두 가지 위치가 있어요.

```
~/.claude/commands/          # 모든 프로젝트에서 사용 (개인용)
프로젝트/.claude/commands/    # 이 프로젝트에서만 사용
```

오늘은 프로젝트 폴더 안에 스킬을 만들어볼 거예요. 먼저 폴더를 만들어보세요.

실습 프로젝트로 이동해서 폴더를 만들어볼게요.

```
cd ~/practice-todo-app
git init
mkdir -p .claude/commands
```

`.claude/commands/` 안에 마크다운 파일을 만들면 그것이 곧 스킬이에요. 파일 이름이 스킬의 슬래시 명령어가 돼요.

파일 이름	실행 방법
`commit.md`	`/commit`으로 실행
`review.md`	`/review`로 실행
`test.md`	`/test`로 실행
`deploy/staging.md`	`/deploy:staging`으로 실행

폴더 안에 하위 폴더를 만들면 콜론으로 구분되는 네임스페이스가 돼요. 관련 스킬을 그룹으로 묶을 때 유용해요.

> 스킬의 핵심은 단순함이에요. 마크다운 파일 하나가 곧 스킬이에요. 별도의 설정 파일이나 등록 과정이 필요 없어요. 파일을 만들면 바로 사용할 수 있어요.

2. 간단한 SKILL.md 작성: 커밋 스킬

2 **첫 번째 스킬을 작성해요: 커밋 자동화**

텍스트 에디터로 커밋 스킬 파일을 만들어보세요.

```
code ~/practice-todo-app/.claude/commands/commit.md
```

스킬 파일의 기본 구조를 알아볼게요. 스킬은 크게 세 부분으로 구성돼요.

스킬 파일의 구조

스킬 파일은 YAML 프론트매터(선택)와 마크다운 본문으로 구성돼요.

```
---
description: "변경사항을 분석하고 커밋 메시지를 작성합니다"
---

# 커밋 스킬

아래 절차를 따라 커밋을 수행하세요.

1. `git status`와 `git diff`를 실행하여 변경사항을 확인하세요
2. 변경사항을 분석하여 적절한 커밋 메시지를 작성하세요
3. 커밋 메시지는 아래 형식을 따르세요:
   - 첫 줄: 타입(feat/fix/docs/refactor): 변경 요약 (50자 이내)
   - 빈 줄
   - 본문: 변경 이유와 내용 (선택)
4. 작성한 커밋 메시지를 사용자에게 보여주고 확인을 받으세요
5. 확인이 되면 `git add`와 `git commit`을 실행하세요
```

이 내용을 `commit.md` 파일에 저장하세요.

[스크린샷: VS Code에서 commit.md 스킬 파일을 작성하는 화면]

각 부분이 무엇을 하는지 살펴볼게요.

부분	설명
프론트매터 ---	스킬의 메타데이터예요. description은 스킬 목록에서 보이는 설명이에요

부분	설명
제목 #	스킬의 이름이에요. Claude가 이 스킬을 로드할 때 참조해요
본문	Claude가 따라야 할 절차와 규칙이에요. 마크다운 형식으로 자유롭게 작성해요

커밋 스킬을 더 정교하게 만들기

기본 버전을 만들었으니 좀 더 정교한 버전으로 업그레이드해볼게요. `commit.md`를 아래 내용으로 교체하세요.

```
---
description: "변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다"
---

# 커밋 스킬

## 절차

1. **변경사항 확인**
   - `git status`로 변경된 파일 목록을 확인하세요
   - `git diff --staged`로 스테이징된 변경사항을 확인하세요
   - 스테이징된 파일이 없으면 `git diff`로 전체 변경사항을 확인하세요

2. **변경사항 분석**
   - 각 파일의 변경 내용을 읽고 "무엇이 바뀌었는지" 파악하세요
   - 변경의 목적(새 기능, 버그 수정, 리팩토링 등)을 판단하세요

3. **커밋 메시지 작성**
   - Conventional Commits 형식을 따르세요:
     - `feat:` 새 기능
     - `fix:` 버그 수정
     - `docs:` 문서 변경
     - `refactor:` 리팩토링
     - `test:` 테스트 추가/수정
     - `chore:` 빌드, 설정 변경
   - 첫 줄은 50자 이내, 한글로 작성
   - 본문이 필요하면 "왜" 변경했는지를 적으세요

4. **확인 및 실행**
   - 작성한 커밋 메시지를 사용자에게 보여주세요
   - 스테이징되지 않은 파일이 있다면 어떤 파일을 포함할지 물어보세요
   - 확인을 받은 후 `git add`와 `git commit`을 실행하세요
```

```
## 규칙
- 커밋 메시지는 반드시 한글로 작성하세요
- 한 커밋에는 하나의 논리적 변경만 포함하세요
- .env, credentials 등 민감 파일은 커밋하지 마세요
```

> 좋은 스킬은 "무엇을 할지"뿐 아니라 "어떤 순서로, 어떤 규칙을 지키면서" 할지를 구체적으로 적어요. Claude는 이 절차를 따라 작업을 수행해요.

3. 트리거 설정과 테스트

3 스킬을 실행하고 동작을 확인해요

스킬 파일을 저장했으면 바로 테스트해볼 수 있어요. Claude Code를 실행하세요.

```
cd ~/practice-todo-app
claude
```

먼저 스킬이 인식되는지 확인해볼게요.

Claude Code에서 `/`를 입력하면 사용 가능한 스킬 목록이 표시돼요.

```
> /
```

방금 만든 `/commit` 스킬이 목록에 보여야 해요. description에 적은 "변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다"가 함께 표시될 거예요.

[스크린샷: Claude Code에서 / 입력 시 스킬 목록이 표시되고 commit 스킬이 보이는 화면]

테스트를 위해 먼저 파일을 수정해볼게요.

```
> src/utils.py에 날짜 포맷팅 함수를 추가해줘
```

Claude가 파일을 수정하면, 이제 커밋 스킬을 실행해보세요.

```
> /commit
```

[스크린샷: /commit 스킬이 실행되어 변경사항을 분석하고 커밋 메시지를 제안하는 화면]

Claude가 스킬에 적힌 절차대로 움직이는 것을 볼 수 있어요.

1. `git status`와 `git diff`를 실행해서 변경사항을 확인하고
2. 변경 내용을 분석해서 타입(feat/fix/docs 등)을 판단하고
3. Conventional Commits 형식으로 커밋 메시지를 작성하고
4. 여러분에게 확인을 요청해요

직접 해보기

커밋 스킬의 동작을 더 깊이 테스트해보세요.

1. 여러 파일을 동시에 수정한 뒤 `/commit`을 실행해보세요. Claude가 변경사항을 어떻게 요약하나요?
2. README.md만 수정한 뒤 `/commit`을 실행해보세요. 타입을 `docs:`로 선택하나요?
3. 버그를 일부러 넣고 수정한 뒤 `/commit`을 실행해보세요. `fix:`를 선택하나요?
4. `.env` 파일을 만들고 수정한 뒤 `/commit`을 실행해보세요. 민감 파일 경고가 나오나요?

스킬에 인자 전달하기

스킬 본문에 `$ARGUMENTS`를 사용하면 사용자가 입력한 추가 텍스트를 받을 수 있어요.

예를 들어 스킬 본문에 이렇게 적으면:

```
사용자가 제공한 추가 지시: $ARGUMENTS
```

실행할 때 이렇게 입력하면:

```
> /commit WIP 커밋이니 메시지 간단하게
```

`$ARGUMENTS` 자리에 “WIP 커밋이니 메시지 간단하게”가 들어가요.

> *`$ARGUMENTS`*를 활용하면 하나의 스킬로 다양한 상황을 처리할 수 있어요. 스킬을 범용적으로 만들면서도 실행 시점에 세부 지시를 전달할 수 있는 거예요.

4. 자동 매칭 확인

4 스킬이 자동으로 매칭되는 메커니즘을 이해해요

스킬은 `/commit`처럼 직접 호출하는 것 외에도, 자연어로 요청하면 Claude가 적절한 스킬을 자동으로 선택해서 실행할 수도 있어요.

이것이 바로 description 필드의 역할이에요. Claude는 사용자의 요청과 각 스킬의 description을 비교해서 가장 적절한 스킬을 찾아요.

자동 매칭을 테스트해볼게요. 슬래시 명령어 없이 자연어로 요청해보세요.

```
> 지금 수정한 내용을 커밋해줘
```

Claude가 `/commit` 스킬을 자동으로 로드해서 실행하는지 확인해보세요.

[스크린샷: 자연어로 커밋을 요청했을 때 Claude가 자동으로 커밋 스킬을 로드하는 화면]

자동 매칭이 잘 되려면 description이 명확해야 해요. 비교해볼게요.

나쁜 description	좋은 description
"커밋"	"변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다"
"코드 관련 작업"	"코드 리뷰를 수행하고 개선 사항을 제안합니다"
"테스트"	"변경된 파일에 대한 단위 테스트를 작성하고 실행합니다"

description이 구체적일수록 Claude가 올바른 스킬을 선택할 확률이 높아져요.

직접 해보기

자동 매칭 실험을 해볼게요. 같은 의도를 다른 표현으로 말해보세요.

1. "변경사항 정리해서 저장해줘", 커밋 스킬이 매칭되나요?
2. "git에 올려줘", 커밋 스킬이 매칭되나요?
3. "코드 수정 마무리해줘", 어떤 스킬이 매칭되나요?

매칭이 잘 안 되는 표현이 있다면, description에 동의어를 추가해보세요. 예를 들어 "변경사항을 분석하고 커밋합니다. git 저장, 코드 올리기"처럼요.

스킬 디버깅 방법

스킬이 예상대로 동작하지 않을 때 디버깅하는 방법을 알아볼게요.

1) 스킬이 목록에 안 보일 때

- 파일 위치가 `.claude/commands/` 안에 있는지 확인하세요
- 파일 확장자가 `.md`인지 확인하세요
- Claude Code를 재시작해보세요

2) 스킬이 로드되는데 절차를 안 따를 때

- 지시사항이 너무 모호하지 않은지 확인하세요
- 단계 번호를 명확히 매기세요
- "반드시", "항상" 같은 강조 표현을 사용하세요

3) 스킬 내용을 수정했는데 반영이 안 될 때

- Claude Code를 완전히 종료하고 다시 시작하세요
- 파일이 정상적으로 저장되었는지 확인하세요

> 스킬은 "Claude에게 주는 지시서"예요. 지시서가 명확할수록 Claude가 정확하게 따라요. 모호한 스킬은 모호한 결과를 만들어요.

5. 정리 및 다음 단계

이번 실습 핵심 정리

- **Skills**는 `.claude/commands/` 폴더의 마크다운 파일이에요
- 파일 이름이 슬래시 명령어가 돼요 (`commit.md` → `/commit`)
- 프론트매터의 description은 자동 매칭에 사용돼요
- 스킬 본문에 절차, 규칙, 형식을 구체적으로 적으면 Claude가 정확하게 따라요
- `$ARGUMENTS`로 실행 시 추가 지시를 전달할 수 있어요

핵심 용어

용어	설명
Skill	Claude Code에게 특정 작업의 수행 방법을 알려주는 마크다운 파일
프론트매터	마크다운 파일 상단의 --- 로 둘러싸인 YAML 메타데이터 영역
슬래시 명령어	/로 시작하는 스킬 실행 명령어. 파일 이름에서 자동 생성
$ARGUMENTS	스킬 실행 시 사용자가 추가로 입력한 텍스트를 받는 변수
자동 매칭	사용자의 자연어 요청을 description과 비교하여 적절한 스킬을 선택하는 기능

트러블슈팅

Q. 스킬 파일을 만들었는데 / 목록에 안 나와요

A. 파일이 `.claude/commands/` 폴더 안에 있는지, 확장자가 `.md`인지 확인하세요. 폴더 이름이 정확히 `commands`인지도 확인하세요 (`command`가 아닌). Claude Code를 재시작해보세요.

Q. 스킬이 실행되는데 절차를 건너뛰어요

A. 스킬 본문의 지시가 모호할 수 있어요. 각 단계를 번호로 명확히 구분하고, '반드시'라는 표현을 추가해보세요. 예: '반드시 git status를 먼저 실행하세요.'

Q. 스킬 내용을 수정했는데 이전 버전으로 동작해요

A. Claude Code를 완전히 종료(`/exit`)하고 다시 시작하세요. 실행 중에 스킬 파일을 수정하면 즉시 반영되지 않는 경우가 있어요.

Q. 자동 매칭이 잘 안 돼요. 엉뚱한 스킬이 실행돼요

A. description을 더 구체적으로 작성하세요. 또한 스킬 수가 적으면 매칭 정확도가 떨어질 수 있어요. 비슷한 스킬이 여러 개일 때는 직접 `/스킬이름`으로 호출하는 것이 확실해요.

실습 체크리스트

- ☐ `.claude/commands/` 폴더를 만들었다
- ☐ `commit.md` 스킬 파일을 작성했다 (프론트매터 + 본문)
- ☐ `/commit`으로 스킬을 실행하고 결과를 확인했다
- ☐ 자연어로 커밋을 요청했을 때 자동 매칭이 되는지 확인했다

– 여기까지 완료하면 이번 실습은 성공이에요! –

- ☐ **[도전] 스킬에 `$ARGUMENTS`를 추가하고 인자를 전달해봤다**
- ☐ **[도전] description을 수정하면서 자동 매칭 정확도 변화를 관찰했다**
- ☐ **[도전] 자기만의 스킬 아이디어를 1개 이상 구상했다**

>> 다음 차시 예고

실습 10차시: Skills 심화: 복합 스킬

하나의 스킬으로는 부족할 때가 있어요. 여러 스킬을 조합하고, 도구 사용 권한을 제어하고, context fork로 독립 실행하는 방법을 배워볼 거예요.